

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Ayala Contreras	26/09/2026
	Nombre: Yader Gonzalo	

Creación de la aplicación para gestión financiera en contenedores en la plataforma Docker

Objetivos

El objetivo principal es guiar a los estudiantes en el aprendizaje y la aplicación de tecnologías de contenedores utilizando la plataforma Docker. La actividad se centra en dotar a los participantes de habilidades prácticas en la creación y la gestión de contenedores, permitiéndoles implementar estas capacidades en el contexto del desarrollo y despliegue automatizado de aplicaciones financieras seguras.

Para ello tendremos estos dos objetivos específicos:

- ▶ Desarrollar experiencia en contenedores Docker: adquirir habilidades prácticas en la creación y la gestión de contenedores Docker utilizando distintos lenguajes de programación, facilitando la portabilidad y consistencia de las aplicaciones.
- ▶ Implementar entornos de desarrollo consistentes: utilizar Docker para configurar entornos de desarrollo replicables y consistentes que garanticen que las aplicaciones se ejecuten de la misma manera en todas las fases, desde desarrollo hasta producción.

Pautas de elaboración

En esta actividad se utilizarán los códigos de las diferentes partes del desarrollo *full stack* del proyecto transversal de la empresa FinTech Solutions S. A., proyecto de desarrollo y despliegue automatizado de una aplicación web segura para gestión financiera, en el ámbito DevOps, y desarrollado por la empresa de desarrollo software TechOps Solutions.

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Ayala Contreras	26/09/2026
	Nombre: Yader Gonzalo	

Deberemos generar, ejecutar y documentar tantos contenedores como se crea necesario en las tecnologías de desarrollo del proyecto FinTech Solutions S. A.

Te dejamos una ayuda: si se crea un aplicativo desplegado en dos capas —aplicativo y base de datos— al menos, serán necesarios dos contenedores; si se realiza un aplicativo con tres capas —presentación, lógica y base de datos— en este caso, serán, al menos, tres contenedores.

Extensión y formato

La entrega consistirá en un archivo ZIP con los ficheros del aplicativo y los ficheros Dockerfile utilizados para la creación de los contenedores que hacen hacer funcionar la aplicación. Se realizará un informe en PDF de máximo quince páginas para dejar constancia de todo lo realizado (sin contar el código incluido en el ZIP). Formato del PDF (Calibri, tamaño 11 e interlineado 1,5).

Rúbrica

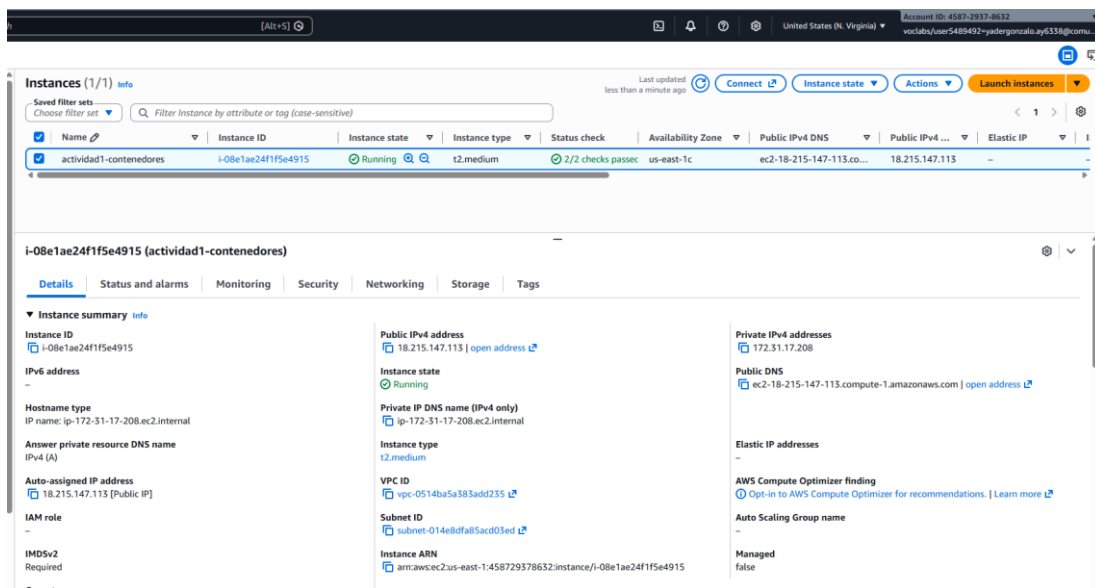
Creación de la aplicación para gestión financiera en contenedores en la plataforma Docker	Descripción	Puntuación máxima (puntos)	Peso %
Criterio 1	Creación de los contenedores.	4	40 %
Criterio 2	Ejecución y guardado de imágenes en repositorio Docker Hub.	2	20 %
Criterio 3	Documentación de la actividad.	4	40 %
		10	100 %

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Ayala Contreras	26/09/2026
	Nombre: Yader Gonzalo	

Desarrollo de la actividad

1. Preparación de la máquina virtual

Se creó una máquina virtual en AWS con sistema operativo Ubuntu 26.04. Esta máquina se utiliza para instalar y configurar Docker, que actúa como motor de ejecución de las imágenes correspondientes al frontend, el backend y la base de datos MongoDB.



2. Acceso a la máquina mediante SSH

Se inició sesión por SSH desde el equipo personal utilizando la clave privada creada para la instancia.

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Ayala Contreras	26/09/2026
	Nombre: Yader Gonzalo	

```
yayala@N100BA5997705:~/unir/contenedores/actividad1/MERN-Stack-Guide/13-Recipe-Book$ ls -ltr
total 24
-rw-r--r-- 1 yayala yayala 2067 Sep 26 11:48 README.md
drwxr-xr-x 2 yayala yayala 4096 Sep 26 12:01 mongodb
drwxr-xr-x 5 yayala yayala 4096 Sep 26 12:01 backend
drwxr-xr-x 4 yayala yayala 4096 Sep 26 12:01 frontend
-rw-r--r-- 1 yayala yayala 777 Sep 26 12:07 docker-compose.yml
drwxr-xr-x 2 yayala yayala 4096 Sep 26 12:09 aws
yayala@N100BA5997705:~/unir/contenedores/actividad1/MERN-Stack-Guide/13-Recipe-Book$ ls -l aws/
total 4
-r----- 1 yayala yayala 1674 Sep 26 12:03 yader-key-aws-unir.pem
yayala@N100BA5997705:~/unir/contenedores/actividad1/MERN-Stack-Guide/13-Recipe-Book$ chmod 400 aws/yader-key-aws-unir.pem
yayala@N100BA5997705:~/unir/contenedores/actividad1/MERN-Stack-Guide/13-Recipe-Book$ ssh -i aws/yader-key-aws-unir.pem ubuntu@18.215.147.113
The authenticity of host '18.215.147.113 (18.215.147.113)' can't be established.
ED25519 key fingerprint is SHA256:H4nGw8FZu6wCgWgc03/QKFjPrPI4ZWeR/1HIWMSVt/Y.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '18.215.147.113' (ED25519) to the list of known hosts.
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Sat Sep 26 17:18:05 UTC 2026

System load:  0.06               Processes:           122
Usage of /:   30.5% of 6.61GB    Users logged in:    0
Memory usage: 6%                IPv4 address for enX0: 172.31.17.208
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
ubuntu@ip-172-31-17-208:~$
```

3. Instalación y configuración de Docker Engine

Para la instalación se utilizó la opción [Install using the convenience script](#), que permite ejecutar el script proporcionado por Docker. Este método no se recomienda para entornos de producción; sin embargo, resulta adecuado para esta actividad académica, cuyo propósito es demostrar la creación y ejecución de imágenes Docker.

Iniciamos descargando el script y ejecutándolo:

```
ubuntu@ip-172-31-17-208:~$ curl -fsSL https://get.docker.com -o get-docker.sh
ubuntu@ip-172-31-17-208:~$ sudo sh get-docker.sh
```

To run Docker as a non-privileged user, consider setting up the Docker daemon in rootless mode for your user:

```
dockerd-rootless-setuptool.sh install
```

Visit <https://docs.docker.com/go/rootless/> to learn about rootless mode.

To run the Docker daemon as a fully privileged service, but granting non-root users access, refer to <https://docs.docker.com/go/daemon-access/>

WARNING: Access to the remote API on a privileged Docker daemon is equivalent to root access on the host. Refer to the 'Docker daemon attack surface' documentation for details: <https://docs.docker.com/go/attack-surface/>

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Ayala Contreras	26/09/2026
	Nombre: Yader Gonzalo	

Después de la instalación, Docker solo podía ejecutarse con privilegios de *root*. Para evitar este uso innecesario de privilegios elevados, se configuró el usuario *ubuntu* para ejecutar Docker como usuario no privilegiado, siguiendo el procedimiento recomendado para Linux.

```
ubuntu@ip-172-31-17-208:~$ dockerd-rootless-setuptool.sh install
[ERROR] Missing system requirements. Run the following commands to
[ERROR] install the requirements and run this tool again.

##### BEGIN #####
sudo sh -eux <<EOF
# Install newuidmap & newgidmap binaries
apt-get install -y uidmap
EOF
##### END #####
```

```
ubuntu@ip-172-31-17-208:~$ sudo apt-get install -y uidmap
```

```
ubuntu@ip-172-31-17-208:~$ dockerd-rootless-setuptool.sh install
[INFO] Creating /home/ubuntu/.config/systemd/user/docker.service
[INFO] starting systemd service docker.service
+ systemctl --user start docker.service
+ sleep 3
+ systemctl --user --no-pager --full status docker.service
+ docker.service - Docker Application Container Engine (Rootless)
Loaded: loaded (/home/ubuntu/.config/systemd/user/docker.service; disabled; preset: enabled)
Active: active (running) since Sat 2026-09-26 17:23:54 UTC; 3s ago
Invocation: 8c851c2d8de5438dbef2767aa8a7e
Docs: https://docs.docker.com/go/rootless/
Main PID: 8019 (rootlesskit)
Tasks: 37
Memory: 62.0M (peak: 62.0M)
CGroup: /user.slice/user-1000.slice/user@1000.service/app.slice/docker.service
└─8019 rootlesskit --state-dir=/run/user/1000/dockerd-rootless --netns=ns0 --slirpnetns-sandbox=auto --slirpnetns-seccomp=auto --disable-host-loopback --port-driver=builtin
└─8027 /proc/self/exe --state-dir=/run/user/1000/dockerd-rootless --netns=ns0 --slirpnetns-sandbox=auto --slirpnetns-seccomp=auto --disable-host-loopback --port-driver=builtin
└─8081 dockerd
└─8081 /usr/bin/containerd --config /run/user/1000/docker/containerd/containerd.toml
```

Finalmente, se realizó una prueba de ejecución para verificar que Docker Engine quedó instalado y funcionando correctamente con el usuario configurado.

```
ubuntu@ip-172-31-17-208:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

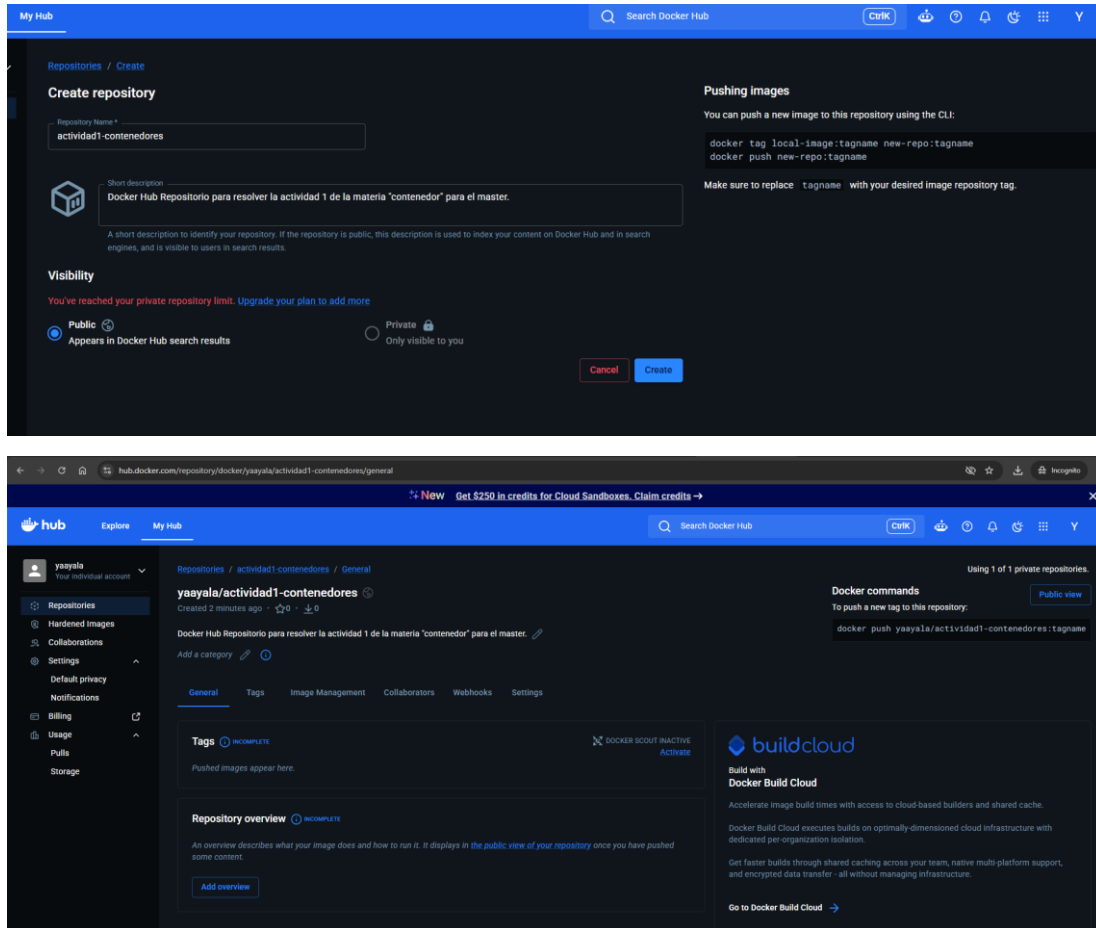
Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Ayala Contreras	26/09/2026
	Nombre: Yader Gonzalo	

4. Configuración del repositorio en Docker Hub

Se inició sesión en [Docker Hub](#) y se creó el repositorio destinado a almacenar las imágenes generadas durante la actividad.



Con la máquina virtual, Docker Engine y Docker Hub configurados, el entorno quedó preparado para contenerizar la aplicación, construir las imágenes, publicarlas en el repositorio y desplegar los contenedores en AWS.

5. Aplicación seleccionada y arquitectura

Se utilizó como base la aplicación [Recipe Book del repositorio MERN Stack Guide](#).

La aplicación original no estaba contenerizada. Por ello, se preparó su ejecución en tres capas independientes: frontend, backend y base de datos MongoDB. Esta separación requiere definir los archivos Dockerfile correspondientes y establecer la comunicación entre los contenedores.

Un **Dockerfile** es un archivo de texto que contiene las instrucciones utilizadas por Docker para construir una imagen de contenedor. Según la [documentación oficial de](#)

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Ayala Contreras	26/09/2026
	Nombre: Yader Gonzalo	

[Docker](#), en él se define la imagen base, los archivos que se copiarán, las dependencias que se instalarán y el comando con el que se iniciará la aplicación.

Por ejemplo, puede indicarle:

1. Qué sistema base usar (Ubuntu, Node.js, Python, etc.).
2. Qué archivos copiar.
3. Qué programas instalar.
4. Qué comando ejecutar cuando la aplicación inicie.

6. Creación de los Dockerfiles

6.1. Frontend

```
frontend > Dockerfile > ...
1  # Etapa 1 (build): imagen de Node.js 24 para compilar la app React
2  FROM node:24-alpine AS build
3
4  # Directorio de trabajo dentro del contenedor
5  WORKDIR /app
6
7  # Copia solo package.json y package-lock.json para aprovechar la caché de capas
8  COPY package*.json ./
9  # Instala todas las dependencias (Vite es necesario para compilar)
10 RUN npm ci
11
12 # Copia el resto del código fuente (excepto lo listado en .dockerignore)
13 COPY . .
14 # Genera los archivos estáticos optimizados en la carpeta dist/
15 RUN npm run build
16
17 # Etapa 2 (runtime): servidor web Nginx ligero para servir los estáticos
18 FROM nginx:alpine
19
20 # Copia solo el resultado del build desde la etapa anterior a la carpeta pública de Nginx
21 COPY --from=build /app/dist /usr/share/nginx/html
22
23 # Configuración de Nginx: sirve la app y reenvía /api y /uploads al backend
24 COPY nginx.conf /etc/nginx/conf.d/default.conf
25
26 # Documenta el puerto en el que escucha Nginx
27 EXPOSE 80
28
29 # Arranca Nginx en primer plano para que el contenedor siga activo
30 CMD ["nginx", "-g", "daemon off;"]
31
```

6.2. Backend

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Ayala Contreras	26/09/2026
	Nombre: Yader Gonzalo	

```

backend > Dockerfile > ...
1  # Imagen base ligera de Node.js 24 sobre Alpine Linux
2  FROM node:24-alpine
3
4  # Directorio de trabajo dentro del contenedor
5  WORKDIR /app
6
7  # Copia solo package.json y package-lock.json para aprovechar la caché de capas
8  COPY package*.json ./
9  # Instala las dependencias exactas del lockfile, sin las de desarrollo
10 RUN npm ci --omit=dev
11
12 # Copia el resto del código fuente (excepto lo listado en .dockerignore)
13 COPY . .
14 # Crea la carpeta de imágenes subidas y se la asigna al usuario node
15 RUN mkdir -p uploads && chown -R node:node uploads
16
17 # Variables de entorno: modo producción, puerto y conexión a MongoDB
18 ENV NODE_ENV=production \
19     PORT=5000 \
20     MONGO_URI=mongodb://mongodb:27017/mern_recipes
21
22 # Ejecuta la app con un usuario sin privilegios (no root)
23 USER node
24
25 # Documenta el puerto en el que escucha la API
26 EXPOSE 5000
27
28 # Comando que arranca el servidor Express
29 CMD ["node", "index.js"]
30

```

6.3. Base de datos MongoDB

```

mongodb > Dockerfile > ...
1  # Imagen base oficial de MongoDB 8.0.3
2  FROM mongo:8.0.3
3
4  # Nombre de la base de datos que se crea al iniciar por primera vez
5  ENV MONGO_INITDB_DATABASE=mern_recipes
6
7  # Documenta el puerto en el que escucha MongoDB
8  EXPOSE 27017
9
10 # Carpeta donde MongoDB guarda los datos (se persiste en un volumen)
11 VOLUME ["/data/db"]
12

```

El código de la aplicación contenerizada y los archivos necesarios para su ejecución se encuentran en el repositorio público:

<https://github.com/yaayala/actividad1-contenedores.git>

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Ayala Contreras	26/09/2026
	Nombre: Yader Gonzalo	

```

ubuntu@ip-172-31-17-208:~/actividad1-contenedores$ docker tag recipe-mongodb yaayala/actividad1-contenedores:mongodb-v1.0
docker tag recipe-backend yaayala/actividad1-contenedores:backend-v1.0
docker tag recipe-frontend yaayala/actividad1-contenedores:frontend-v1.0
ubuntu@ip-172-31-17-208:~/actividad1-contenedores$ docker image ls

IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
hello-world:latest                  5e2309035332                      25.9kB      9.49kB        U
recipe-backend:latest               805367d656c1                      273MB       67.7MB
recipe-frontend:latest              2e3d134205b2                      93.8MB      26.4MB
recipe-mongodb:latest               58e776a6a1a3                      1.27GB      315MB
yaayala/actividad1-contenedores:backend-v1.0  805367d656c1                      273MB       67.7MB
yaayala/actividad1-contenedores:frontend-v1.0  2e3d134205b2                      93.8MB      26.4MB
yaayala/actividad1-contenedores:mongodb-v1.0  58e776a6a1a3                      1.27GB      315MB
ubuntu@ip-172-31-17-208:~/actividad1-contenedores$ docker push yaayala/actividad1-contenedores:mongodb-v1.0
docker push yaayala/actividad1-contenedores:backend-v1.0
docker push yaayala/actividad1-contenedores:frontend-v1.0
The push refers to repository [docker.io/yaayala/actividad1-contenedores]
edd1ed89f0d4: Pushed
87d0ea48540d: Pushed
44136fa355b3: Pushed
5e68fb960432: Pushed
3bc1468bbf2a: Pushed
adf7abd1fe48: Pushed
8d07092de802: Pushed
051f5efcc90f: Pushed
1aea085c9ccb: Pushed
730272032256: Pushed
mongodb-v1.0: digest: sha256:58e776a6a1a3fe0b2ea5e2de04e6f6e944ce932fa342c745a14e013a7875f584 size: 856
The push refers to repository [docker.io/yaayala/actividad1-contenedores]
c6a7daf0f420: Pushed
44136fa355b3: Already exists
3c500d5a707b: Pushed
e2de96513ba9: Pushed
02f40d53b06f: Pushed
7ce7dcba7b7f: Pushed
464a44f64ac7: Pushed
b02712882ee2: Pushed
580f7b0db38d: Pushed
4887f070bdf: Pushed
acbad198f37a: Pushed
backend-v1.0: digest: sha256:805367d656c1bb03d5439d9b0e9327261c77f3a55758b995b71f233a54c3502c size: 856
The push refers to repository [docker.io/yaayala/actividad1-contenedores]
171d7992906a: Pushed
44136fa355b3: Already exists
d9aae54b5831: Pushed
e2de96513ba9: Layer already exists

```

Resultado:

Repositorios / actividad1-contenedores / General

yaayala/actividad1-contenedores

Last pushed 1 minute ago · Repository size: 388.6 MB · 0 · 0

Docker Hub Repositorio para resolver la actividad 1 de la materia "contenedor" para el master.

Add a category

General
Tags
Image Management
Collaborators
Webhooks
Settings

Tags

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
frontend-v1.0		Image	less than 1 day	1 minute
backend-v1.0		Image	less than 1 day	1 minute
mongodb-v1.0		Image	less than 1 day	1 minute

See all

Docker Scout Inactive
Activate

buildcloud

Build with Docker Build Cloud

Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

Get faster builds through shared caching across your team, native multi-platform support, and encrypted data transfer - all without managing infrastructure.

Go to Docker Build Cloud

Docker commands

To push a new tag to this repository:

docker push yaayala/actividad1-contenedores:
Copy

Using 1 of 1 private repositories.

Actividades

10

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Ayala Contreras	26/09/2026
	Nombre: Yader Gonzalo	

```
ubuntu@ip-172-31-17-148:~/actividad1-contenedores$ docker pull yaayala/actividad1-contenedores:mongodb-v1.0
docker pull yaayala/actividad1-contenedores:backend-v1.0
docker pull yaayala/actividad1-contenedores:frontend-v1.0
mongodb-v1.0: Pulling from yaayala/actividad1-contenedores
Digest: sha256:ddd9b5f519cd01db765634532c988faa29b9a6bbe284e7961cde773bd73ba394
Status: Image is up to date for yaayala/actividad1-contenedores:mongodb-v1.0
docker.io/yaayala/actividad1-contenedores:mongodb-v1.0
backend-v1.0: Pulling from yaayala/actividad1-contenedores
Digest: sha256:805367d656c1bb03d5439d9b0e9327261c77f3a55758b995b71f233a54c3502c
Status: Image is up to date for yaayala/actividad1-contenedores:backend-v1.0
docker.io/yaayala/actividad1-contenedores:backend-v1.0
frontend-v1.0: Pulling from yaayala/actividad1-contenedores
Digest: sha256:2e3d134205b23eff90a2ae71a3fc1e8b13610de7c7c0dbb7235eaf8879313c0b
Status: Image is up to date for yaayala/actividad1-contenedores:frontend-v1.0
docker.io/yaayala/actividad1-contenedores:frontend-v1.0
ubuntu@ip-172-31-17-148:~/actividad1-contenedores$ docker image ls
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
yaayala/actividad1-contenedores:backend-v1.0	805367d656c1	273MB	67.7MB	
yaayala/actividad1-contenedores:frontend-v1.0	2e3d134205b2	93.8MB	26.4MB	
yaayala/actividad1-contenedores:mongodb-v1.0	ddd9b5f519cd	1.18GB	296MB	

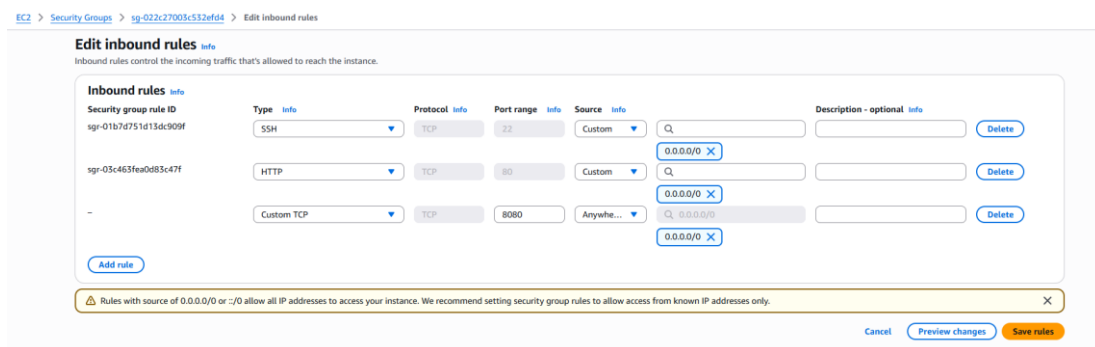
8. Red de contenedores y ejecución de la solución

Antes de iniciar los contenedores se creó la red **recipe-net** mediante el comando **docker network create recipe-net**. Esta red permite que frontend, backend y MongoDB se comuniquen mediante sus nombres de contenedor.

```
ubuntu@ip-172-31-17-148:~/actividad1-contenedores$ docker run -d --name backend --network recipe-net -p 5000:5000 \
-e MONGO_URI=mongodb://mongodb:27017/mern_recipes \
-v uploads:/app/uploads \
yaayala/actividad1-contenedores:backend-v1.0
d0c8374774783b1e9cc5d542e5b24388c1c25fd2238c1cc27cb3174ede93a6
ubuntu@ip-172-31-17-148:~/actividad1-contenedores$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
d0c837477478	yaayala/actividad1-contenedores:backend-v1.0	"docker-entrypoint.s..."	3 seconds ago	Up 2 seconds	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp	backend
1d9fb59dd7c4	yaayala/actividad1-contenedores:mongodb-v1.0	"docker-entrypoint.s..."	4 minutes ago	Up 3 minutes	27017/tcp	mongodb
93dd1868dbdc5f543991274ce36dd64a68a1ab092bd2bdece7c74db67a99478a	yaayala/actividad1-contenedores:frontend-v1.0	"docker-entrypoint.s..."	2 seconds ago	Up 2 seconds	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	frontend
d0c837477478	yaayala/actividad1-contenedores:backend-v1.0	"docker-entrypoint.s..."	17 seconds ago	Up 16 seconds	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp	backend
1d9fb59dd7c4	yaayala/actividad1-contenedores:mongodb-v1.0	"docker-entrypoint.s..."	4 minutes ago	Up 4 minutes	27017/tcp	mongodb

Como el frontend se publicó a través del puerto 8080, se ajustó el grupo de seguridad de la instancia para permitir las conexiones entrantes necesarias por ese puerto.



La evidencia final muestra la aplicación en funcionamiento sobre los contenedores Docker desplegados en la máquina virtual de AWS.

Asignatura	Datos del alumno	Fecha
Contenedores	Apellidos: Ayala Contreras	26/09/2026
	Nombre: Yader Gonzalo	

Not secure 98.89.48.66:8080

My Recipe Book

Add New Recipe

Frijoles rojos, carne molida de res, chicharrón de cerdo, chorizo colombiano, arroz blanco, huevo, plátano maduro, aguacate, arepa antioqueña, tomate, cebolla larga, ajo, comino, sal, pimienta, aceite vegetal, agua

7. Corte los plátanos maduros en tajadas y tráelos hasta que estén dorados.
8. Prepare huevos fritos con la yema al gusto.
9. Caliente las arepas antioqueñas en una plancha o sartén.
10. Sirva en una bandeja amplia una porción de frijoles, arroz, carne molida, chicharrón, chorizo, huevo frito, plátano maduro, arepa y aguacate.
11. Acompañe con el resto del guiso criollo sobre los frijoles o la carne, si lo desea.
12. Servir caliente.

Recipe Image:

Not secure 98.89.48.66:8080



Bandeja Paisa Colombiana Tradicional

Ingredients: Frijoles rojos, carne molida de res, chicharrón de cerdo, chorizo colombiano, arroz blanco, huevo, plátano maduro, aguacate, arepa antioqueña, tomate, cebolla larga, ajo, comino, sal, pimienta, aceite vegetal, agua

Instructions: 1. Remoje los frijoles rojos en agua durante toda ...

Conclusión. Esta actividad me permitió la creación, configuración y preparación de un entorno Docker en una virtual machine de AWS, se logró aprender la contenerización de una aplicación de tres capas, su publicación de imágenes y el despliegue funcional en AWS. Esto permite tener una solución separada por componentes aislados que trabajan juntos ofreciendo una funcionalidad.